Information Processing and Management 62 (2025) 104232

Contents lists available at ScienceDirect

Information Processing and Management

journal homepage: www.elsevier.com/locate/ipm

AFR-Rank: An effective and highly efficient LLM-based listwise reranking framework via filtering noise documents

Yinghao Xiong, Xinhui Tu *, Weizhong Zhao

Hubei Provincial Key Laboratory of Artificial Intelligence and Smart Learning, Wuhan, China
National Language Resources Monitoring and Research Center for Network Media, Wuhan, China
School of Computer Science, Central China Normal University, Wuhan, China

ARTICLE INFO

Keywords:
Document reranking
Noise documents filtering
Large Language Models

ABSTRACT

Recently, listwise reranking methods powered by Large Language Models (LLMs) have demonstrated promising results across multiple retrieval benchmarks. However, these approaches often suffer from substantial noise introduced by irrelevant documents from first-stage retrieval, posing significant challenges to the subsequent reranking process. Such noise not only degrades ranking performance but also increases computational overhead. To address this issue, we propose AFR-Rank, an innovative LLM-driven reranking framework that systematically filters irrelevant documents prior to reranking. The framework consist of three essential components: an Assessor, a Filter, and a Reranker. The Assessor first evaluates the relevance of each document retrieved from the initial stage. Documents deemed highly relevant are then processed by the Filter, which further eliminates noisy candidates to produce a refined candidate set. Finally, the Reranker generates the ultimate ranked list from these selected candidates. Notably, the parallel architecture of the Assessor and Filter ensures high efficiency throughout the framework. Comprehensive experiments conducted on widely used benchmarks, including TREC DL and BEIR, demonstrate that AFR-Rank consistently outperforms existing state-of-the-art methods in terms of NDCG@10. Our efficiency evaluations on the TREC DL benchmark reveal that AFR-Rank achieves approximately 2.7 times improvement in efficiency compared to RankGPT while significantly reducing total LLM API costs. Additional ablation studies and comparative analyses further confirm AFR-Rank's strong generalizability and robustness across diverse query scenarios.

1. Introduction

Document reranking is a core task in information retrieval, aiming to assess the relevance of the top-k documents (typically k= 100) retrieved by a first-stage retriever (e.g., BM25 (Robertson, Zaragoza, et al., 2009)) and generate the list of documents with refined order (Yuan, 2024; Zhai, 2024). Notably, LLM-based document reranking methods exhibit superior performance in zero-shot settings, significantly surpassing traditional supervised reranking models (Cho, 2023; Drozdov, 2023; Liu, Zhu, & Dou, 2024; Sun, Zhuang, Wang, & Zuccon, 2025). Existing LLM-based approaches can be divided into three categories, pointwise, pairwise and listwise, according to the number of documents processed simultaneously (Zhu et al., 2024).

Since the listwise approaches consider multiple documents simultaneously in a single inference, they achieve an excellent trade-off between effectiveness and efficiency (Chen et al., 2025). Therefore, we focus on listwise document reranking in this study.

* Corresponding author.
E-mail address: tuxinhui@ccnu.edu.cn (X. Tu).

https://doi.org/10.1016/j.ipm.2025.104232
Received 3 April 2025; Received in revised form 20 May 2025; Accepted 24 May 2025
Available online 16 June 2025
0306-4573/© 2025 Elsevier Ltd. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

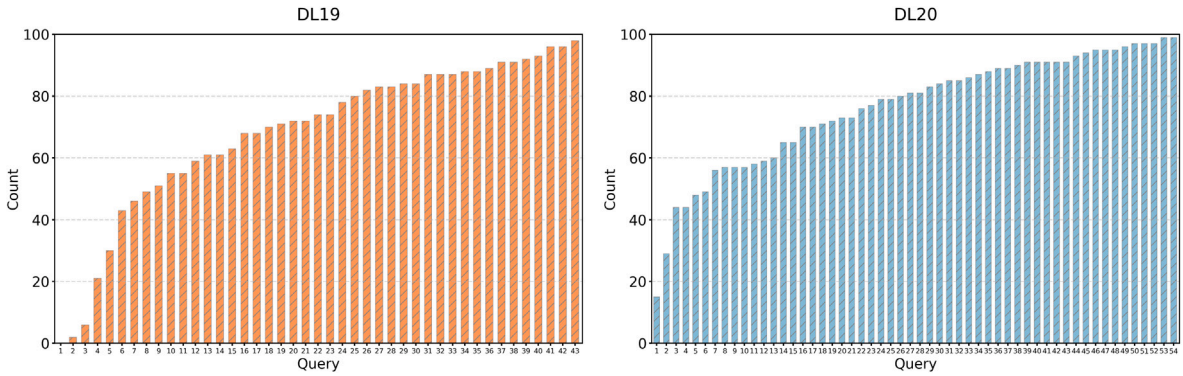

Fig. 1. The number of irrelevant documents in the top-100 documents obtained by BM25 on DL19 and DL20 datasets.

Table 1
Average number of irrelevant documents in the top-100 documents for queries on the TREC DL datasets and sex BEIR datasets. Note that we test the results obtained by using BM25, Contriever, and SPLADE++ED as first-stage retriever, respectively.

Method	DL19	DL20	Covid	NFCorpus	Touche	DBPedia	SciFact	Signal
BM25	68.09	76.35	52.82	61.66	90.00	90.37	98.84	92.69
Contriever	63.72	70.93	59.30	92.13	92.96	89.02	98.93	93.39
SPLADE++ED	60.88	67.13	43.94	92.26	92.20	88.12	98.94	93.16

Although widely used in industrial applications, such as web search engines (Zhang, Hofstätter, Lewis, Tang, & Lin, 2025), the listwise approach still has the following drawback: a substantial amount of noise in retrieval results poses a great challenge to the reranking process. As illustrated in Fig. 1, for the TREC DL19 and DL20 datasets (Craswell, Mitra, Yilmaz, & Campos, 2021; Craswell, Mitra, Yilmaz, Campos, & Voorhees, 2020), nearly half of queries retrieve more than 80 irrelevant documents in the top-100 documents from the first-stage results. This issue is even more severe on multiple BEIR datasets (Thakur, Reimers, Rücklé, Srivastava, & Gurevych, 2021).

To investigate the number of noise documents in first-stage retrieval results further, we test three different first-stage retrievers, including BM25, Contriever (Izacard et al., 2022), and SPLADE++ED (Formal, Lassance, Piwowarski, & Clinchant, 2022). BM25 is a classical lexical retrieval method that ranks documents based on term frequency and inverse document frequency statistics. Contriever is a dual-encoder dense retrieval model trained with contrastive learning, which maps queries and documents into a shared semantic space. SPLADE++ED enhances sparse retrieval by learning vocabulary expansion through BERT-based document and query encoders, producing interpretable yet effective sparse representations. The detailed statistics are presented in Table 1. It can be observed that for TREC DL datasets, regardless of the retriever used, on average, more than 60% of the top-100 documents for each query are irrelevant. For Signal and SciFact, the ratio of irrelevant documents is greater than 90%. Furthermore, although using advanced retrievers can reduce noise to some extent, the amount of noise remains substantial, significantly diminishing the effectiveness of the subsequent reranking stage.

Existing listwise reranking methods typically need to evaluate a significant number of irrelevant documents, resulting in suboptimal ranking performance and increased computational overhead. Recent studies in filtering strategies commonly adopt a scoring-based approach, estimating the relevance of documents in initial retrieval results for a given query (Nouriinanloo & Lamothe, 2024). Documents are then selected for reranking based on a predefined relevance threshold. However, depending solely on relevance scores is inadequate for achieving an optimal initial candidate ordering for reranking. Thus, developing a more robust and efficient filtering mechanism to effectively exclude noise documents before reranking is critical.

In this paper, we propose a novel LLM-based listwise reranking framework called AFR-Rank, which systematically filters irrelevant documents prior to reranking. Our framework includes three key processing stages: (1) **Relevance Assessment**: The top-100 documents retrieved by the first-stage retriever are partitioned into multiple groups. Each group is evaluated by the Assessor, which assigns a relevance score to each document. After processing all groups, only documents with high relevance scores enter the next stage. (2) **Noise Filtering**: The selected documents are further grouped based on their relevance scores and the original ranking order within the first-stage retrieval. For each group, the Filter ranks the documents in descending order of relevance to the query. Finally, the top-20 most relevant documents are sequentially sampled from all groups to form the candidate documents. (3) **Document Reranking**: The Reranker employs a sliding window strategy to generate the final ranking results.

It is worth noting that the grouping strategy enables parallel processing of the Assessor and Filter, significantly improving the efficiency of our framework. Furthermore, to reduce both query latency and computational cost, our framework employs a collaborative mechanism that integrates large-scale and lightweight LLMs. In the Relevance Assessment and Noise Filtering stages, we utilize a lightweight LLM as the base model, since scoring and ranking documents within a moderate-sized group are relatively straightforward tasks. As for the Document Reranking stage, we employ a large-scale LLM to handle the challenging task that requires considering larger context windows.

To summarize, the key contributions of this paper are listed as follows:

- We introduce AFR-Rank, a novel LLM-based listwise reranking framework designed to systematically enhance retrieval perfermence by filtering irrelevant documents before reranking. AFR-Rank employs an innovative two-stage approach, termed "assess-then-filter", to effectively identify and eliminate irrelevant documents from initial retrieval results.
- AFR-Rank significantly reduces query latency and computational demands by integrating parallel processing and a collaborative mechanism that leverages both large-scale and lightweight language models.
- Extensive evaluations conducted on the TREC DL datasets and 6 BEIR datasets reveal that AFR-Rank consistently outperforms state-of-the-art methods, particularly demonstrated by notable improvements in NDCG@10 metrics. Further ablation studies and comparative analyses underscore AFR-Rank's robustness and generalizability, validating its effectiveness across a diverse range of query scenarios.

2. Related work

2.1. Document reranking with LLMs

Recent studies have increasingly focused on employing LLMs to rerank documents. Broadly speaking, existing LLM-based reranking approaches can be classified into three major categories: pointwise, pairwise, and listwise (Huang & Chen, 2024; Liang et al., 2023; Luo, Chen, He, & Sun, 2024; Parry, MacAvaney, & Ganguly, 2024; Pradeep, Sharifymoghaddam, & Lin, 2023; Rathee, MacAvaney, & Anand, 2025; Sachan et al., 2022; Tang, Zhang, Ma, Lin, & Ture, 2024).Liang et al. (2023) and Sachan et al. (2022) explored two representative pointwise paradigms—binary relevance generation and query generation—to assess the relevance of query–document pairs. In binary relevance generation, the LLM is prompted to output a simple "YES" or "NO" based on whether each document is relevant to a given query, and the probability of generating "YES" (or "NO") serves as the document's relevance score. Extending this idea, Zhuang, Qin et al. (2024) introduced a more fine-grained relevance labeling scheme, demonstrating that incorporating granular labels can significantly improve the effectiveness of LLM-based rerankers. For the query generation approach, a zero-shot question generation model is employed to score the relevance between a query and a retrieved document. Specifically, a pretrained language model computes the probability of the query token sequence conditioned on the text of the retrieved document. This technique enables rich cross-attention interactions between query and document, while avoiding additional domain-specific training.

Qin (2024) introduced the Pairwise Ranking Prompting (PRP) paradigm, in which the prompt includes both the query and a pair of candidate documents. The LLM must then decide which document is more relevant and output the corresponding label, e.g., "Passage A" or "Passage B". A notable advantage of PRP is its applicability to both scoring-oriented LLMs and generative LLMs, since the output space is restricted to two discrete options. However, PRP typically incurs substantial computational and latency costs, given that it requires pairwise comparisons across many document pairs. To alleviate the computational overhead associated with PRP, Zhuang, Zhuang, Koopman, and Zuccon (2024) refined this paradigm through a Setwise comparison approach. Rather than performing one pairwise comparison at a time, the Setwise method prompts the LLM to select the most relevant document from a set of candidates. While Setwise effectively reduces the number of pairwise inferences, it still demands considerable resources when handling a large set of documents

Among these, the listwise approach achieves a remarkable trade-off between effectiveness and efficiency by simultaneously considering multiple documents in a single inference. Specifically, Ma, Zhang, Pradeep, and Lin (2023) and Sun et al. (2023) initially explored listwise document reranking using commercial closed-source LLMs, and proposed LRL and RankGPT. However, due to the inherently sequential nature of the sliding window strategy, both LRL and RankGPT suffer from a significant issue of inefficiency. In order to address the inefficiency, Chen et al. (2025) proposed TourRank, which mitigates to some extent the problem of the sliding window strategy. However, it introduces additional drawback that the computational cost is relatively high, due to requiring multiple round of selection and introducing a large number of irrelevant documents in each round.

2.2. Relevance evaluation with LLMs

The ability to reliably assess the relevance of retrieved documents has long been central to the development and validation of information retrieval systems (Liusie, Raina, Fathullah, & Gales, 2024; Yan et al., 2024). Traditional human-based assessments, while considered a gold standard, are expensive, difficult to scale, and frequently subject to inconsistencies due to annotator bias or fatigue (Chiang & Lee, 2023b; Dewan, Liu, & Shah, 2025). In light of these limitations, a growing body of work has begun exploring the potential of LLMs as an alternative or complementary means of conducting relevance judgments (Li et al., 2024; Shankar, 2024; Takehi, 2025; Upadhyay et al., 2024).

Chiang and Lee (2023a) represent one of the earliest systematic efforts to evaluate whether LLMs can replicate—or even surpass—the quality of expert human annotations. Their study employed a straightforward methodology in which the same instructions, evaluation samples, and questions given to human annotators were presented to an LLM. The model was instructed to assign a quality score using a 5-point Likert scale, and the generated textual responses were subsequently parsed to extract a single numerical rating. Remarkably, the results indicated a high level of agreement between LLM-generated scores and those obtained through expert human judgment. More recently, Faggioli et al. (2023) proposed an extended framework that investigates various ways in which LLMs could assist or partner with human assessors in relevance determination. They characterized a set of human–machine collaboration strategies, spanning from minimal reliance on machine output to scenarios in which humans primarily confirm or refine LLM-provided judgments. In their empirical evaluation, Faggioli compared LLM-based and human-based assessments using

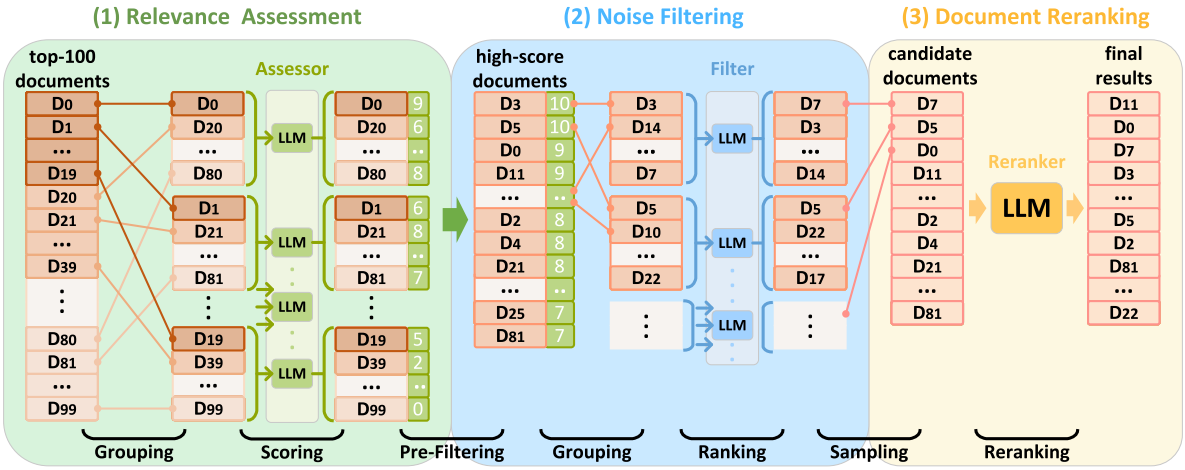

Fig. 2. The architecture of AFR-Rank, which consists of three stages: (1) Relevance Assessment, (2) Noise Filtering and (3) Document Reranking.

the TREC-8 and TREC DL21 collections, revealing both promising alignments and notable divergences, thus offering preliminary but valuable insights into the reliability and effectiveness of LLM-driven relevance judgments.

A subsequent wave of research has broadened these initial inquiries. Several studies examine the capability of LLMs to handle incomplete or otherwise imperfectly annotated test collections. For example, Abbasiantaeb and Meng (2024) investigated the application of LLMs to fill judgment gaps in conversational search benchmarks. Their findings indicate that while fine-tuned LLaMA models can achieve stronger alignment with human labels, systematic biases may emerge depending on the models' configurations and scoring behaviors. In parallel, Mehrdad et al. (2024) explored the potential of fine-tuned LLMs in e-commerce settings. Leveraging a specialized dataset of query–item pairs, they demonstrated that models optimized with LoRA techniques can achieve alignment with human evaluations at approximately 89% parity, thereby significantly curbing annotation time and expenses. Further highlighting the versatility of LLM-based evaluations, researchers have also tested zero-shot scenarios in which off-the-shelf LLMs serve directly as relevance assessors. Zhang, Yu, et al. (2023) proposed an architecture that conceptualizes LLMs as individual "neurons" within a layered scoring network, yielding notable gains on multiple benchmarks. Complementary to this, Raina, Liusie, and Gales (2024) probed the robustness of zero-shot LLM-based assessments by introducing adversarial perturbations, thereby exposing the potential vulnerabilities and biases embedded in these models. Both studies employ a "LLM-scoring" paradigm, prompting the model to produce a direct quantitative assessment of item quality, typically on a fixed scale spanning from 0 to k.

2.3. Noise documents filtering

Existing filtering approaches typically employ a scoring model to estimate the relevance of documents in the first-stage results for a given query (Nouriinanloo & Lamothe, 2024). A predefined threshold is then used to select documents with high relevance score for subsequent reranking. For instance, Chang et al. (2024) proposed a noise filtering mechanism within a Retrieval-Augmented Generation(RAG) framework, introducing a training-free approach where multiple LLM agents collaboratively filter and rank retrieved documents. However, only relying on relevance scores is insufficient for establishing an optimal initial candidates order for the reranker. Moreover, assessing relevance in a strictly sequential manner further increases query latency.

3. Methodology

As illustrated in Fig. 2, our proposed AFR-Rank consists of three crucial stages: (1) **Relevance Assessment**: We introduce an Assessor to evaluate the relevance of the top-100 documents to a given query. Only documents identified as highly relevant is advanced to the next stage. (2) **Noise Filtering**: The selected documents undergo further filtering through a Filter, forming candidate documents with an optimized ranking order. (3) **Document Reranking**: The Reranker employs a sliding window strategy to generate the final ranking results. In this section, we provide a detailed description of each stage.

3.1. Relevance assessment

In the first stage, we introduce an Assessor which consists of multiple LLMs, to evaluate the relevance of the top-100 documents to a given query. Traditional LLM-based evaluation methods typically adopt a pointwise paradigm, generating a relevance score for each query-document pair independently. Such process necessitates repeated inference across all retrieved documents. To improve

both the effectiveness and efficiency of the assessment process, we employ a listwise approach and propose a novel prompting strategy, termed ***listwise scoring***, to assess documents collectively rather than individually.

In the ***listwise scoring*** strategy, the prompt includes a list of documents alongside the given query. The LLM is then instructed to produce a structured JSON output, in which each key corresponds to a document identifier and each value denotes a relevance score on a scale from 0 to k. In this study, we set the maximum relevance score k to 10. The treatment of enforcing a structured output format can substantially reduce the probability of erroneous outputs. Detailed prompt specifications are provided in Appendix A.1.

To further accelerate the assessment process, we incorporate a grouping mechanism into our framework, as illustrated in Algorithm 1. Specifically, the top-100 documents retrieved by the first-stage retriever are divide into $\boldsymbol{n}$ groups, with each group containing five documents (which is the predefined group size). By evenly distributing the documents in the initial order across these groups, we make sure that there exist documents with different relevance scores in each group. Liu, Lin, et al. (2024) found that current LLMs do not robustly access and use information in long input contexts because of the position bias. In order to reduce the impact of the position bias in LLM-based evaluations, we shuffle the documents within each group before passing them to the Assessor.

By combining the effective prompting strategy with this grouping mechanism, the Assessor is capable of processing multiple groups in parallel, significantly improving the efficiency of relevance assessment.

3.2. Noise filtering

The objective of this stage is to select the documents with high relevance to the given query, thereby constructing the candidate list for the subsequent Document Reranking stage. Specifically, after receiving the assessment results, we first filter out documents which are deemed as irrelevant or only partially relevant by the Assessor. Because the Assessor employs a scoring strategy, a straightforward approach is to define a threshold and remove all documents with scores below that threshold. The detailed analysis of how the threshold setting impacts the filtering performance is provided in Section 4.5.

After that, we first rank high-score documents in descending order according to their relevance scores. For documents sharing the same score, we employ their relative order in the original top-100 retrieval results as a secondary criterion. We then apply the grouping mechanism from the Relevance Assessment stage, partitioning these documents into $\boldsymbol{m}$ groups. In practice, the number of high-score documents for a given query may not be the multiple of the predefined group size. For this case, we distribute them as evenly as possible to guarantee the filtering performance. As the Filter considers only high-score documents, the total number of groups at this stage is considerably less than that in the Relevance Assessment stage.

For each group, we prompt the LLM with documents in the group alongside the given query, instructing it to produce a ranking of the documents with the descending order of the relevance to the query. To reduce computational cost, we employ a lightweight LLM as base model for both Assessor and Filter. However, existing prompting strategies are ill-suited to moderate-sized open-source models for ranking task, often resulting in formatting errors. To address this limitation, we propose a novel ***group ranking*** prompting strategy specifically tailored for lightweight models, which significantly reduces the incidence of erroneous outputs. Furthermore, we shuffle the documents within each group before providing them to the Filter to mitigate position bias. The detailed prompt used in this process are provided in Appendix A.2.

After the Filter generates the ranking results, we sequentially sample candidate documents from all groups. Specifically, if more than 20 high-score documents are obtained, we sample only the top-20 candidates; otherwise, we collect all documents across groups. The detailed sampling process is outlined in Algorithm 2.

This two-stage filtering mechanism termed "assess-then-filter" ensures the construction of the ordered candidate list for the Reranker, ultimately enhancing reranking performance.

3.3. Document reranking

In this stage, we employ a large-scale LLM for document reranking. Specifically, we adopt a sliding window strategy with the window size of 10 and the stride of five. The detailed prompt is provided in Appendix A.3.

Compared to traditional sliding window-based reranking approaches, our Reranker operates only on the top-20 candidate documents, thereby substantially reducing the computational overhead of serial processing. Furthermore, for tasks under strict time constraints, the Document Reranking stage can be skipped, outputting the candidate documents obtained from the Noise Filtering stage as the final results. Experiments in Section 4.2 indicate that, after the processing of the first two stages, the obtained ranking of candidate documents is acceptable for applications.

Algorithm 1: Grouping Algorithm

```
Input: Document list L, group size s
Output: Groups G
if |L| mod s == 0 then
    Number of Groups n ← |L| // s
else
    Number of Groups n ← |L| // s + 1
Groups G ← [ [ ] for _ in range(n) ]
for i ← 0 to |L| − 1 do
    reminder r ← i mod n
    Append L[i] to G[r]
return G
```

Algorithm 2: Sampling Algorithm

Input: Groups G, number of groups n
Output: Candidate list D
high-score documents count $c \leftarrow \sum_{G_i \in G} |G_i|$
if $c > 20$ **then**
  candidate documents count $d \leftarrow 20$
else
  candidate documents count $d \leftarrow c$
Candidate list $D \leftarrow [\,]$
for $i \leftarrow 0$ ***to*** $d - 1$ **do**
  reminder $r \leftarrow i \bmod n$
  element $e \leftarrow G[r].\text{pop}(0)$
  Append e to D
return D

4. Experiments

To comprehensively evaluate the performance of the proposed AFR-Rank method, we design an extensive series of experiments focused on answering the following critical research questions:

RQ1: Does our proposed AFR-Rank method outperform current state-of-the-art baseline approaches across various datasets?
RQ2: What is the efficiency advantage of AFR-Rank compared to traditional non-parallel methods?
RQ3: How effective is our newly proposed "assess-then-filter" methodology in comparison to existing filtering methods?
RQ4: How do variations in hyperparameter settings influence the ranking performance of AFR-Rank?
RQ5: To what extent does AFR-Rank maintain robustness across diverse query scenarios?

4.1. Experimental settings

Datasets. To assess the effectiveness of our proposed framework, we conduct a series of experiments on the TREC DL and BEIR benchmarks. TREC DL is used to evaluate the task of ad-hoc retrieval on large-scale Web collections. We employ the 2019 and 2020 test sets: TREC-DL19 and TREC-DL20, which include 43 and 54 queries, respectively. Both use the MS MARCO v1 passage corpus, which contains 8.8 million passages. The BEIR benchmark comprises a diverse collection of information retrieval tasks, enabling comprehensive evaluation across different domains. Owing to resource constraints, we select 6 representative datasets from BEIR: Covid, NFCorpus, Touche, DBPedia, SciFact, and Signal for our experiments.

The datasets presented cover a wide array of domains and retrieval scenarios, each designed to evaluate information retrieval methods under different conditions and knowledge requirements. The Covid dataset leverages the COVID-19 Open Research Dataset (CORD-19) and primarily focuses on medical research queries related to COVID-19. NFCorpus is a medical information retrieval dataset composed of clinical notes, medical literature, and health information websites, designed to evaluate retrieval system performance on health-related queries. The Touche dataset is specifically designed for argument retrieval tasks, containing text from various online forums and debate platforms. DBPedia is tailored to factual query retrieval concerning entities, drawing its corpus from Wikipedia articles and structured knowledge bases. SciFact is specifically constructed to verify scientific claims through queries requiring confirmation or refutation by consulting relevant research papers. Lastly, the Signal dataset, sourced from Signal AI, encompasses one million news articles and blog posts in multiple languages. Each dataset, thus, uniquely contributes to a comprehensive understanding of retrieval efficacy across specialized, structured, scientific, and multilingual information landscapes. Detailed statistical information on the evaluation datasets used is provided in Table 2.

Baselines and metrics. We compare our framework against several state-of-the-art supervised and unsupervised document reranking methods. The supervised baselines include: (1) monoBERT (Nogueira & Cho, 2020), a BERT-large cross-encoder reranker trained on the MS MARCO dataset; (2) monoT5 (Nogueira, Jiang, Pradeep, & Lin, 2020), a sequence-to-sequence reranker that leverages the T5 model to compute relevance scores. The unsupervised baselines include two listwise approaches: (1) RankGPT; (2) TouRank, and two pointwise approaches: (1) DIRECT(0, 10) (Zhuang, Qin et al., 2024); (2) MCRanker (Guo et al., 2025). RankGPT is regarded as a strong LLM-based zero-shot document reranking approach, employing a sliding window strategy to perform listwise reranking. TouRank introduces a tournament-based ranking mechanism, achieving state-of-the-art performance on multiple datasets while maintaining a reasonable computational cost. DIRECT(0, 10) prompts the LLM to assign relevance scores from 0 to 10 for each query-document pair, and then ranking documents in descending order of scores. As for MCRanker, it generates ranking scores based on multiple criteria, substantially improving the performance of pointwise LLM rerankers. For a fair comparison with previous studies, we use BM25 (via Pyserini[1] with default settings) as the first-stage retriever to produce the top-100 documents. We measure effectiveness in terms of NDCG@10.

[1] https://github.com/castorini/pyserini

Table 2
Statistical characteristics of the datasets. For each dataset, we analyze key metrics including the number of queries, corpus size, count of annotated documents, and the average number of relevant documents per query in the corpus.

Dataset	Domain	Queries	Corpus	Qrels	Rel D/Q
DL19	Web	43	8.8M	9.3K	95.4
DL20	Web	54	8.8M	11.4K	66.8
Covid	Biomedical	50	171K	66.3K	493.5
NFCorpus	Biomedical	323	3.6K	12.3K	38.2
Touche	Argument	49	382K	0.9K	19.0
DBPedia	Encyclopedia	400	4.6M	43.5K	38.2
SciFact	Science	300	5K	0.3K	1.1
Signal	Social Media	97	2.9M	1.9K	19.6

Table 3
Performance (NDCG@10) comparison of different methods on TREC DL and BEIR benchmarks. We reproduce RankGPT with gpt-4o. Best performing unsupervised and overall methods are marked bold. †For AFR-Rank (w/o Reranker), we use the candidate documents as final results directly. The version of gpt-4o is gpt-4o-2024-08-06, while the version of gpt-4o-mini is gpt-4o-mini-2024-07-18.

Method	DL19	DL20	Covid	NFCorpus	Touche	DBPedia	SciFact	Signal	BEIR (Avg)
BM25	50.58	47.96	59.47	30.75	**44.22**	31.80	67.89	**33.05**	44.53
Supervised Methods									
monoBERT (340M)	70.50	67.28	70.01	36.88	31.75	41.87	71.36	31.44	47.22
monoT5 (220M)	71.48	66.99	78.34	37.38	30.82	42.42	73.40	31.67	49.01
monoT5 (3B)	71.83	68.89	80.71	38.97	32.41	44.45	**76.57**	32.55	**50.94**
Unsupervised Methods									
DERECT(0, 10) (gpt-4)	–	–	79.30	37.52	25.22	40.82	70.08	29.12	47.01
MCRanker (gpt-4)	–	–	**84.23**	**39.58**	32.48	**44.67**	73.14	29.73	50.64
TourRank-10 (gpt-3.5-turbo)	71.63	**69.56**	82.59	37.99	29.98	44.64	72.17	30.83	49.70
RankGPT (gpt-4o)	73.76	66.98	81.60	37.30	**33.52**	44.09	74.49	30.98	50.33
AFR-Rank w/o † (gpt-4o-mini)	71.57	66.37	79.30	35.68	32.89	40.09	70.94	32.58	48.58
AFR-Rank (gpt-4o-mini+gpt-4o)	**74.14**	69.20	83.38	37.55	32.94	42.44	**75.11**	**32.99**	**50.74**

Implementation details. Following the setting in Guo et al. (2025), we set the maximum relevance score k to 10 during the Relevance Assessment stage. In the Noise Filtering stage, the threshold is a critical hyperparameter which directly influences the final ranking performance. To identify the most effective threshold, for each dataset, we first apply the Filter module to extract candidate documents using threshold values ranging from nine to five, and the candidates derived by the threshold yielding the highest NDCG@20 score are advanced to the next stage, i.e., the Reranker. Owing to resource constraints, gpt-4o-mini is employed as the base model for both the Assessor and Filter modules, while gpt-4o is utilized as the base model for the Reranker.

4.2. Main results (RQ1)

Table 3 reports the performance of all evaluated methods on TREC DL datasets and 6 BEIR datasets. Our proposed framework achieves the best results on DL19, SciFact, and Signal, and also attains the highest average performance on the BEIR datasets. Specifically, we draw the following key conclusions:

1. AFR-Rank surpasses all baselines on DL19 and shows competitive, albeit slightly suboptimal, performance on DL20. Notably, TourRank-10 performs reranking ten times and aggregates the outcomes to produce the final ranking, resulting in substantially higher query latency. In addition, AFR-Rank achieves a 2.31% gain in terms of NDCG@10 on DL19, compared to monoT5 (3B) which is a state-of-the-art supervised baseline.
2. Across the 6 BEIR datasets, AFR-Rank demonstrates the best overall performance. Note that both the Signal and SciFact datasets contain relatively large-scale noise in the top-100 documents retrieved for each query. Specifically, the average number of relevant documents per query retrieved by BM25 is only 7 for Signal, while for SciFact, just one relevant document in the entire corpus. As for the Signal dataset where many baselines perform poorly, our approach achieves superior ranking effectiveness. Moreover, on the SciFact dataset, AFR-Rank attains an NDCG@10 score of 75.11, substantially outperforming other unsupervised baselines. Overall, AFR-Rank achieves the highest average performance among all unsupervised methods across these 6 BEIR tasks, indicating its effectiveness and suitability for domain-specific retrieval scenarios.
3. Furthermore, our results indicate that even without the Reranker, AFR-Rank's advanced filtering mechanism delivers performance which is comparable with or better than that of several strong baselines. This characteristic is especially valuable for applications with strict latency constraints, wherein exhaustive reranking may be computationally infeasible.

Table 4
Efficiency comparison between RankGPT and AFR-Rank on TREC DL19 and DL20 datasets. The comparison metrics include query latency (measured in seconds) and computational cost (measured in USD).

Methods	DL19		DL20	
	latency (s)	cost ($)	latency (s)	cost ($)
RankGPT	61.77	3.01	67.61	4.00
AFR-Rank	25.21	1.15	22.74	1.34

Table 5
Filtering performance (P@20 and NDCG@20) comparison of assess-only and assess-then-filter mechanisms. We conduct experiments with three different backbone LLMs, including Gemma2-9B, Phi4-14B, and gpt-4o-mini.

Methods	Backbones	DL19		DL20	
		P@20	NDCG@20	P@20	NDCG@20
BM25	–	54.42	49.14	46.94	47.21
assess-only	Gemma2-9B	63.14	56.74	51.94	51.24
assess-then-filter (ours)		**65.21**	**61.51**	**53.89**	**56.21**
assess-only	Phi4-14B	66.28	61.79	56.85	58.61
assess-then-filter (ours)		**66.74**	**63.11**	**59.17**	**60.56**
assess-only	gpt-4o-mini	69.42	65.20	58.43	60.49
assess-then-filter (ours)		**69.53**	**66.49**	**59.07**	**62.99**

4.3. Evaluation of efficiency (RQ2)

In this section, we conduct a comparative experiment on DL19 and DL20 to evaluate the efficiency of the proposed framework. Specifically, we compare AFR-Rank with RankGPT in terms of query latency and computational cost. For each dataset, the query latency is measured by the average processing time (in second) of the whole reranking process in each method, while the computational cost denotes the total cost (in US dollar) of LLM API usage for each method. The results are presented in Table 4.

Table 4 demonstrates a clear efficiency improvement of AFR-Rank over RankGPT according to both query latency and computational cost on both two datasets. Specifically, on DL19, AFR-Rank achieves a 2.45 times efficiency improvement over RankGPT, i.e., the average query latency decreases from 61.77s to 25.21s. Moreover, the total cost of LLM API usage for AFR-Rank drops drastically from $3.01 to $1.15. Note that the results on DL20 exhibits a similar pattern. The above observations indicate that AFR-Rank provides a promising solution to real-world applications.

4.4. Impact of filtering mechanism (RQ3)

To evaluate the effectiveness of our "assess-then-filter" strategy, we compare the obtained performance under different backbone models, including Gemma2-9B, Phi4-14B, and gpt-4o-mini. For comparison, baselines, which employ simply a threshold to filter out irrelevant documents according to the relevance scores obtained by the assessment process, are denoted by "assess-only" in this section.

Table 5 presents the evaluation results on DL19 and DL20, showing that our approach considerably outperforms traditional assess-only filtering across all tested models. The performance gains are particularly significant on moderate-sized open-source models. Specifically, under Gemma2-9B, the obtained NDCG@20 improves by 4.77 and 4.97 on DL19 and DL20, respectively, while under gpt-4o-mini, the improvements are 1.29 and 2.50 for the same datasets, respectively.

Furthermore, when using gpt-4o-mini, our method outperforms the original BM25 baseline in terms of P@20 by 27.77% on DL19 and 25.84% on DL20. Even with Gemma2-9B, the corresponding improvements are 19.83% and 14.81%, respectively. These results demonstrate that our assess-then-filter strategy achieves effective filtering performance.

4.5. Impact of diverse thresholds (RQ4)

In this part, we investigate how different settings of threshold affect both candidate documents quality and processing latency. Specifically, the threshold is adjusted from nine down to five for DL19 and DL20, while keeping k as constant 10. The reason why we do not use the threshold 10 is that it would excessively limit the number of documents entering the Noise Filtering stage. Additionally, we find that documents with score less than five are irrelevant with high probability. Therefore, we do not consider cases where the threshold is below five. The experimental results are presented in Fig. 3.

From Fig. 3, we can find that our model with setting threshold eight derives the best trade-off between effectiveness (in terms of NDCG@20) and efficiency (in terms of Latency). The main reason is that setting too big threshold (eg. nine) might filter out too many relevant documents, which will degrade the quality of derived candidates. On the contrary, when setting a relatively small threshold, in the Noise Filtering stage, more documents need to be handled, leading to the high query latency.

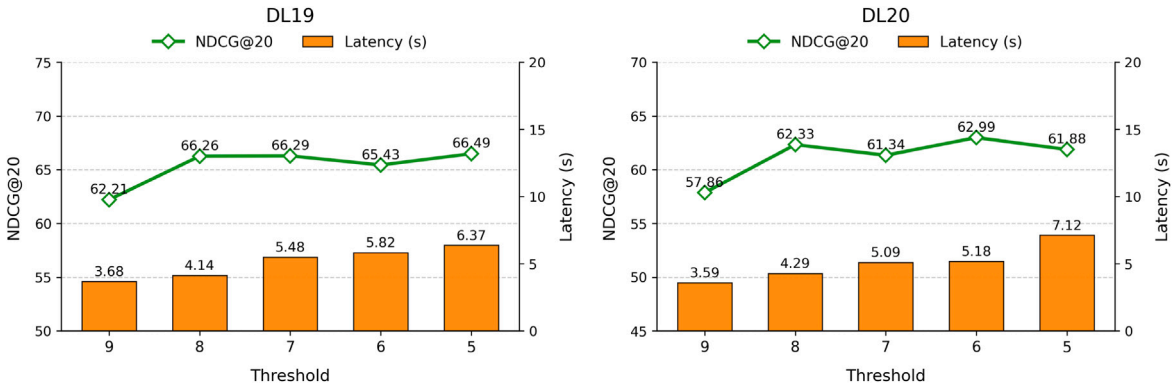

Fig. 3. Evaluation results of varying thresholds on NDCG@20 of candidate documents and processing latency for DL19 and DL20 using gpt-4o-mini. The reported latency is the average processing time per query in the Noise Filtering stage.

Table 6
Performance (NDCG@5 and NDCG@10) comparison of AFR-Rank with different number of candidate documents.

	DL19		DL20	
	NDCG@5	NDCG@10	NDCG@5	NDCG@10
Top-10	77.33	73.41	69.96	67.12
Top-15	77.69	**74.66**	71.39	68.02
Top-20	**78.64**	74.14	**72.03**	**69.20**
Top-25	76.07	72.89	71.85	68.62
Top-30	75.61	73.01	68.81	68.66

We also evaluate our approach on 6 BEIR datasets, the results of which are not presented here due to the space limit. As anticipated, for more challenging tasks (e.g., Signal) where the corpus contains a significant fraction of brief texts and unlabeled documents, the optimal threshold proves to be five. As for the dataset like Covid, which contains a sufficient relevant documents, setting the threshold to eight yields a higher performance. These findings provide practical guidelines for threshold selection based on dataset characteristics.

4.6. Impact of the number of candidates (RQ4)

The number of candidate documents which entering Reranker is a important hyperparameter in the whole framework. For investigating the impact of candidate number, we test the final ranking performance with adjusting this parameter from 10 to 30 (in a step five). Results are presented in the Table 6.

From Table 6, we can find that AFR-Rank achieves the best average performance when using the top-20 candidate documents in Reranker on DL19 and DL20. Specifically, using less candidates (i.e., 10 or 15) might miss highly relevant documents, hurting the final performance. On the contrary, employing too many candidates (i.e., 25 or 30) might introduce extra irrelevant documents, deteriorating the final performance as well. Due to resource constraints, we consistently employs the top-20 candidate documents for reranking in our experiments.

4.7. Impact of initial ranking (RQ5)

To assess the robustness of our approach for various initial rankings, we test the performance of AFR-Rank with three different input orders: (1) BM25, the original top-100 documents retrieved by BM25, (2) RandomBM25, a random permutation of the BM25 ranking, and (3) InverseBM25, the reverse order of BM25. Specifically, taking gpt-4o-mini as backbone LLM, we compare the performance of RankGPT and our framework.

The results in Fig. 4 reveal that RankGPT is highly sensitive to the initial ranking order, whereas our framework demonstrates stable performance across all scenarios. The robustness of AFR-Rank arises from its sophisticated filtering mechanism, wherein the grouping and sampling strategies collectively ensure a globally optimal ranking performance.

4.8. Impact of different first-stage retrieval models (RQ5)

To investigate how AFR-Rank performs with various first-stage retrieval methods, we conduct experiments on AFR-Rank with BM25, Contriever (Izacard et al., 2022), and SPLADE++ED (Formal et al., 2022). In our experiments, gpt-4o-mini is utilized as the base model for both Assessor and Filter, while gpt-4o is employed as the base model for Reranker.

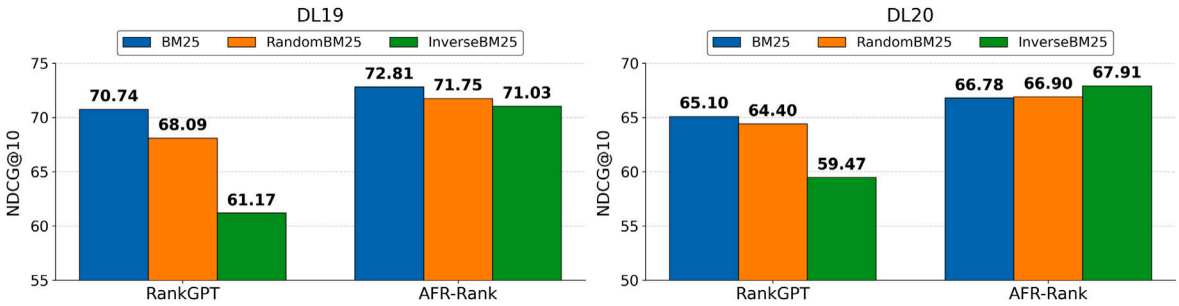

Fig. 4. Sensitivity analysis results of the initial ranking for RankGPT and AFR-Rank.

Table 7
Performance (nDCG@10) comparison of AFR-Rank with different first-stage retrievers on DL19 and DL20.

Method	DL19	DL20
BM25	50.58	47.96
BM25 → AFR-Rank	74.14	69.20
Improvement	+46.58%	+44.29%
Contriever	62.02	63.42
Contriever → AFR-Rank	72.38	71.60
Improvement	+16.70%	+12.90%
SPLADE++ ED	73.08	71.97
SPLADE++ ED → AFR-Rank	76.28	72.79
Improvement	+4.38%	+1.14%

As shown in Table 7, our framework benefits significantly from increasingly effective retrievers, consistently improving its overall performance. Note that AFR-Rank performs slightly worse on DL19 when using Contriever, compared to AFR-Rank with BM25. This phenomenon is also observed in TourRank's results. The possible reason lies in that underlying factors of Contriever will affect the final ranking performance, which needs to be further investigated as future work.

5. Discussion and implication

5.1. Implications

This study presents AFR-Rank, an novel listwise reranking method that significantly outperforms traditional approaches. AFR-Rank integrates a sophisticated post-scoring strategy, combining document grouping, precise ranking, and strategic sampling. This method effectively identifies and removes irrelevant documents, enhancing retrieval quality substantially. Comprehensive experiments on diverse datasets and multiple baseline models consistently validate AFR-Rank's superior performance, marking it as a notable advancement in reranking technology.

In addition to theoretical progress, the paper provides valuable insights for practical deployment, especially regarding the optimization of large language model (LLM)-based reranking in industrial applications. It demonstrates that parallel processing and the strategic collaboration between large-scale and lightweight LLMs significantly reduce query latency and resource consumption without sacrificing result quality. Given that high latency remains a considerable barrier for real-world implementation, these findings offer practical guidance for more efficient and viable reranking solutions, paving the way for broader industrial adoption.

5.2. Limitations

Due to the inherent variability of LLM outputs, changing the input order of documents may produce different relevance scores assigned to a given document. Accordingly, improving the stability and reliability of relevance scoring remains a key direction for enhancing the performance of our framework. We discuss this issue in Appendix B. Furthermore, although we offer empirically derived guidelines for threshold selection in the Noise Filtering stage, dynamically adapting the threshold to each specific query will be a promising research topic for future work. We provide further discussion on query-adaptive threshold setting methods in Appendix C. Lastly, AFR-Rank with a more advanced LLM could potentially achieve a substantial performance gain. Nevertheless, such investigation requires more extensive computational resources.

6. Conclusions

This paper presents AFR-Rank, a novel three-stage listwise reranking framework that systematically filters irrelevant documents

prior to reranking. Firstly, an Assessor is used to evaluate the relevance of the first-stage results to a given query. Then, a Filter is employed to filter the irrelevant documents out based on the Assessor's output. Finally, a Reranker produces the final ranking results. Experimental results on several widely used datasets demonstrate the effectiveness of our approach. In addition, the parallel processing strategy can significantly improve the efficiency of the proposed framework. The results of comprehensive experiments suggest that AFR-Rank is a feasible solution to real-world applications.

CRediT authorship contribution statement

Yinghao Xiong: Methodology, Investigation, Conceptualization, Writing – original draft. **Xinhui Tu:** Project administration, Supervision, Funding acquisition, Writing – review & editing. **Weizhong Zhao:** Funding acquisition, Writing – review & editing.

Acknowledgments

This work was supported by the National Natural Science Foundation of China (Grant No. 62472192) and the Hubei Province Education Science Planning 2023 Special Funding Key Project (Grant No. 2023ZA018): Research on Digital Transformation and Innovation of Teaching Models in Higher Education.

Appendix A. Prompts

A.1. Listwise scoring prompt

The specific prompt used for listwise scoring is present here.

Listwise scoring

There is a query and {num} documents. Each document has a unique docno.

```
{
  "<QUERY>": {query},

  "<DOCUMENTS>":[
    {"<docno>": "D0", "<content>": {document0}},
    {"<docno>": "D1", "<content>": {document1}},
    ...]
}
```

Please assign a score for each document to illustrate its relevance for the query. Score ranging from 0 to 10. Your answer must be a json data where the key is the docno and the value is the score, without any other content! Such as { 'D0': 5, 'D1': 8, 'D2': 7, 'D3': 9...}.

A.2. Group ranking prompt

The specific prompt used for group ranking is present here.

Group ranking

There is a query and {num} documents. Each document has a unique docno.

```
{
  "<QUERY>": {query},

  "<DOCUMENTS>":[
    {"<docno>": "D0", "<content>": {document0}},
    {"<docno>": "D1", "<content>": {document1}},
    ...]
}
```

Please rank these {num} documents in descending order based on their relevance to the query. Ranking the documents by docno as follows format in your answer, such as: [D3, D1...]. Do not provide explanations or any additional commentary and make sure your answer only have {num} above docons in the list.

Table 8
Performance (NDCG@20 and P@20) comparison with different scoring strategies on DL19 and DL20.

Methods	DL19		DL20	
	NDCG@20	P@20	NDCG@20	P@20
Iter1	66.75	70.12	62.52	59.91
Iter2	65.95	69.30	60.89	58.24
Iter3	66.78	69.88	60.88	58.15
Mean	**68.61**	**71.40**	**63.78**	60.93
Max	66.75	70.47	61.72	58.98
Min	67.50	70.58	63.72	**61.48**

A.3. Document reranking prompt

The specific prompt used for document reranking is present here.

Document reranking

There is a query and {num} documents. Each document has a unique docno.

```
{
  "<QUERY>": {query},

  "<DOCUMENTS>":[
    {"<docno>": "D0", "<content>": {document0}},
    {"<docno>": "D1", "<content>": {document1}},
    ...]
}
```

Please rank these {num} documents in descending order based on their relevance to the query. List the documents by docno as follows format in your answer, such as: [D4]>[D1]>[D8]... Do not provide explanations or any additional commentary and make sure your answer only have {num} unique docons in the list.

Appendix B. Discussion of different scoring strategies

We conducted additional experiments on the DL19 and DL20 datasets to investigate ways to mitigate the instability of LLM output scores. Specifically, we ran the Assessor three times on each input and measured the NDCG@20 and P@20 of the document ranking produced after each run. Since the data fed into the Assessor is shuffled each time, the document order differs across the three runs. We also evaluated three aggregation strategies—using the mean, the maximum, and the minimum of the three scores. The detailed results are shown in the Table 8.

From the Table 8, it is clear that the rankings produced by individual runs can differ substantially. For example, on the DL20 dataset the difference between the first and third runs is almost 1.5 points in both NDCG@20 and P@20, indicating that LLM scoring does exhibit noticeable fluctuations. We further observe that taking the average of the three runs as the final score significantly improves ranking performance, demonstrating that this simple aggregation strategy can help stabilize LLM scores.

Appendix C. Discussion of different threshold strategies

To further explore query-adaptive threshold setting methods, we designed the following two new threshold setting strategies and conducted experimental validation on DL19 and DL20.

Strategy 1, "LLM-based Threshold": We performed statistical analysis on the scores obtained from the Relevance Assessment stage. We then provided these statistical results to the LLM and asked it to suggest a threshold based on the score distribution. Finally, the LLM returns a threshold that it considers capable of distinguishing between relevant and irrelevant documents.

Strategy 2. For documents entering the Noise Filtering stage, we always take the top-50 documents, we call this the "Top-50 Threshold". We chose 50 because our analysis on DL19 and DL20 shows that, when using either the empirical threshold or Strategy 1, an average of 50 documents per query reach the Noise Filtering stage.

The experimental results in Table 9 show that using the LLM-based threshold setting strategy performs better on the DL20 dataset than the empirical method we used in the paper. However, on the DL19 dataset, the empirical method achieves a higher NDCG@10 score. This indicates that query-adaptive threshold setting methods are a strategy worth continuing to explore, and we will conduct more detailed research in our future work.

Table 9
Performance (NDCG@10 and P@10) comparison with different threshold setting strategies on DL19 and DL20.

Methods	DL19		DL20	
	NDCG@10	P@10	NDCG@10	P@10
Empirical Threshold	**74.14**	82.09	69.20	72.78
Top-50 Threshold	71.88	80.00	67.29	71.67
LLM-based Threshold	72.35	**83.49**	**69.81**	**74.07**

Data availability

The data that support the findings of this study are available from the corresponding author upon reasonable request.

References

Abbasiantaeb, Z., & Meng, C. (2024). Can we use large language models to fill relevance judgment holes? Retrieved from https://arxiv.org/abs/2405.05600.

Chang, C.-Y., Jiang, Z., Rakesh, V., Pan, M., Yeh, C.-C. M., Wang, G., et al. (2024). MAIN-RAG: Multi-agent filtering retrieval-augmented generation. Retrieved from https://arxiv.org/abs/2501.00332.

Chen, Y., Liu, Q., Zhang, Y., Sun, W., Ma, X., Yang, W., et al. (2025). Tourrank: utilizing large language models for documents ranking with a tournament-inspired strategy. In *WWW '25*, *Proceedings of the ACM on web conference 2025* (pp. 1638–1652). New York, NY, USA: Association for Computing Machinery, ISBN: 9798400712746, http://dx.doi.org/10.1145/3696410.3714863.

Chiang, C.-H., & Lee, H.-y. (2023a). Can large language models be an alternative to human evaluations? In A. Rogers, J. Boyd-Graber, & N. Okazaki (Eds.), *Proceedings of the 61st annual meeting of the association for computational linguistics (volume 1: long papers)* (pp. 15607–15631). Toronto, Canada: Association for Computational Linguistics, http://dx.doi.org/10.18653/v1/2023.acl-long.870, https://aclanthology.org/2023.acl-long.870/.

Chiang, C.-H., & Lee, H.-y. (2023b). A closer look into using large language models for automatic evaluation. In H. Bouamor, J. Pino, & K. Bali (Eds.), *Findings of the Association for Computational Linguistics: EMNLP 2023* (pp. 8928–8942). Singapore: Association for Computational Linguistics, http://dx.doi.org/10.18653/v1/2023.findings-emnlp.599, https://aclanthology.org/2023.findings-emnlp.599/.

Cho, S. (2023). Discrete prompt optimization via constrained generation for zero-shot re-ranker. In A. Rogers, J. Boyd-Graber, & N. Okazaki (Eds.), *Findings of the Association for Computational Linguistics: ACL 2023* (pp. 960–971). Toronto, Canada: Association for Computational Linguistics, http://dx.doi.org/10.18653/v1/2023.findings-acl.61, https://aclanthology.org/2023.findings-acl.61/.

Craswell, N., Mitra, B., Yilmaz, E., & Campos, D. (2021). Overview of the TREC 2020 deep learning track. CoRR, abs/2102.07662. Retrieved from https://arxiv.org/abs/2102.07662.

Craswell, N., Mitra, B., Yilmaz, E., Campos, D., & Voorhees, E. M. (2020). Overview of the TREC 2019 deep learning track. CoRR, abs/2003.07820. Retrieved from https://arxiv.org/abs/2003.07820.

Dewan, M., Liu, J., & Shah, C. (2025). LLM-driven usefulness labeling for IR evaluation. Retrieved from https://arxiv.org/abs/2503.08965.

Drozdov, A. (2023). PaRaDe: passage ranking using demonstrations with LLMs. In H. Bouamor, J. Pino, & K. Bali (Eds.), *Findings of the Association for Computational Linguistics: EMNLP 2023* (pp. 14242–14252). Singapore: Association for Computational Linguistics, http://dx.doi.org/10.18653/v1/2023.findings-emnlp.950, https://aclanthology.org/2023.findings-emnlp.950/.

Faggioli, G., Dietz, L., Clarke, C. L. A., Demartini, G., Hagen, M., Hauff, C., et al. (2023). Perspectives on large language models for relevance judgment. In *ICTIR 201923*, *Proceedings of the 2023 ACM SIGIR international conference on theory of information retrieval* (pp. 39–50). ACM, http://dx.doi.org/10.1145/3578337.3605136.

Formal, T., Lassance, C., Piwowarski, B., & Clinchant, S. (2022). From distillation to hard negative sampling: making sparse neural IR models more effective. In *SIGIR '22*, (pp. 2353–2359). New York, NY, USA: Association for Computing Machinery, ISBN: 9781450387323, http://dx.doi.org/10.1145/3477495.3531857.

Guo, F., Li, W., Zhuang, H., Luo, Y., Li, Y., Yan, L., et al. (2025). MCRanker: generating diverse criteria on-the-fly to improve pointwise LLM rankers. In *WSDM '25*, *Proceedings of the eighteenth ACM international conference on web search and data mining* (pp. 944–953). New York, NY, USA: Association for Computing Machinery, ISBN: 9798400713293, http://dx.doi.org/10.1145/3701551.3703583.

Huang, C.-W., & Chen, Y.-N. (2024). InstUPR : Instruction-based unsupervised passage reranking with large language models. Retrieved from https://arxiv.org/abs/2403.16435.

Izacard, G., Caron, M., Hosseini, L., Riedel, S., Bojanowski, P., Joulin, A., et al. (2022). Unsupervised dense information retrieval with contrastive learning. Retrieved from https://arxiv.org/abs/2112.09118.

Li, H., Dong, Q., Chen, J., Su, H., Zhou, Y., Ai, Q., et al. (2024). LLMs-as-judges: A comprehensive survey on LLM-based evaluation methods. Retrieved from https://arxiv.org/abs/2412.05579.

Liang, P., Bommasani, R., Lee, T., Tsipras, D., Soylu, D., Yasunaga, M., et al. (2023). Holistic evaluation of language models. Retrieved from https://arxiv.org/abs/2211.09110.

Liu, N. F., Lin, K., Hewitt, J., Paranjape, A., Bevilacqua, M., Petroni, F., et al. (2024). Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics*, *12*, 157–173. http://dx.doi.org/10.1162/tacl_a_00638, Retrieved from https://aclanthology.org/2024.tacl-1.9/.

Liu, W., Zhu, Y., & Dou, Z. (2024). DemoRank: Selecting effective demonstrations for large language models in ranking task. Retrieved from https://arxiv.org/abs/2406.16332.

Liusie, A., Raina, V., Fathullah, Y., & Gales, M. (2024). Efficient LLM comparative assessment: a product of experts framework for pairwise comparisons. In Y. Al-Onaizan, M. Bansal, & Y.-N. Chen (Eds.), *Proceedings of the 2024 conference on empirical methods in natural language processing* (pp. 6835–6855). Miami, Florida, USA: Association for Computational Linguistics, http://dx.doi.org/10.18653/v1/2024.emnlp-main.389, https://aclanthology.org/2024.emnlp-main.389/.

Luo, J., Chen, X., He, B., & Sun, L. (2024). PRP-graph: Pairwise ranking prompting to LLMs with graph aggregation for effective text re-ranking. In L.-W. Ku, A. Martins, & V. Srikumar (Eds.), *Proceedings of the 62nd annual meeting of the association for computational linguistics (volume 1: long papers)* (pp. 5766–5776). Bangkok, Thailand: Association for Computational Linguistics, http://dx.doi.org/10.18653/v1/2024.acl-long.313, Retrieved from https://aclanthology.org/2024.acl-long.313/.

Ma, X., Zhang, X., Pradeep, R., & Lin, J. (2023). Zero-shot listwise document reranking with a large language model. Retrieved from https://arxiv.org/abs/2305.02156.

Mehrdad, N., Mohapatra, H., Bagdouri, M., Chandran, P., Magnani, A., Cai, X., et al. (2024). Large language models for relevance judgment in product search. Retrieved from https://arxiv.org/abs/2406.00247.

Nogueira, R., & Cho, K. (2020). Passage re-ranking with BERT. Retrieved from https://arxiv.org/abs/1901.04085.

Nogueira, R., Jiang, Z., Pradeep, R., & Lin, J. (2020). Document ranking with a pretrained sequence-to-sequence model. In T. Cohn, Y. He, & Y. Liu (Eds.), *Findings of the Association for Computational Linguistics: EMNLP 2020* (pp. 708–718). Association for Computational Linguistics, http://dx.doi.org/10.18653/v1/2020.findings-emnlp.63, https://aclanthology.org/2020.findings-emnlp.63/.

Nouriinanloo, B., & Lamothe, M. (2024). Re-ranking step by step: Investigating pre-filtering for re-ranking with large language models. Retrieved from https://arxiv.org/abs/2406.18740.

Parry, A., MacAvaney, S., & Ganguly, D. (2024). Top-down partitioning for efficient list-wise ranking. Retrieved from https://arxiv.org/abs/2405.14589.

Pradeep, R., Sharifymoghaddam, S., & Lin, J. (2023). RankVicuna: Zero-shot listwise document reranking with open-source large language models. Retrieved from https://arxiv.org/abs/2309.15088.

Qin, Z. (2024). Large language models are effective text rankers with pairwise ranking prompting. In K. Duh, H. Gomez, & S. Bethard (Eds.), *Findings of the Association for Computational Linguistics: NAACL 2024* (pp. 1504–1518). Mexico City, Mexico: Association for Computational Linguistics, http://dx.doi.org/10.18653/v1/2024.findings-naacl.97, https://aclanthology.org/2024.findings-naacl.97/.

Raina, V., Liusie, A., & Gales, M. (2024). Is LLM-as-a-judge robust? investigating universal adversarial attacks on zero-shot LLM assessment. In Y. Al-Onaizan, M. Bansal, & Y.-N. Chen (Eds.), *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing* (pp. 7499–7517). Miami, Florida, USA: Association for Computational Linguistics, http://dx.doi.org/10.18653/v1/2024.emnlp-main.427, https://aclanthology.org/2024.emnlp-main.427/.

Rathee, M., MacAvaney, S., & Anand, A. (2025). Guiding retrieval using llm-based listwise rankers. In *Advances in information retrieval: 47th european conference on information retrieval, ECIR 2025, Lucca, Italy, April 6–10, 2025, proceedings, part I* (pp. 230–246). Berlin, Heidelberg: Springer-Verlag, ISBN: 978-3-031-88707-9, http://dx.doi.org/10.1007/978-3-031-88708-6_15.

Robertson, S., Zaragoza, H., et al. (2009). The probabilistic relevance framework: BM25 and beyond. *Foundations and Trends® in Information Retrieval*, *3*(4), 333–389.

Sachan, D., Lewis, M., Joshi, M., Aghajanyan, A., Yih, W.-t., Pineau, J., et al. (2022). Improving passage retrieval with zero-shot question generation. In Y. Goldberg, Z. Kozareva, Y. Zhang (Eds.), *Proceedings of the 2022 conference on empirical methods in natural language processing* (pp. 3781–3797). Abu Dhabi, United Arab Emirates: Association for Computational Linguistics, http://dx.doi.org/10.18653/v1/2022.emnlp-main.249, https://aclanthology.org/2022.emnlp-main.249/.

Shankar (2024). Who validates the validators? aligning llm-assisted evaluation of llm outputs with human preferences. In *Proceedings of the 37th annual ACM symposium on user interface software and technology* (pp. 1–14).

Sun, W., Yan, L., Ma, X., Wang, S., Ren, P., Chen, Z., et al. (2023). Is ChatGPT good at search? Investigating large language models as re-ranking agents. In H. Bouamor, J. Pino, K. Bali (Eds.), *Proceedings of the 2023 conference on empirical methods in natural language processing* (pp. 14918–14937). Singapore: Association for Computational Linguistics, http://dx.doi.org/10.18653/v1/2023.emnlp-main.923, https://aclanthology.org/2023.emnlp-main.923/.

Sun, S., Zhuang, S., Wang, S., & Zuccon, G. (2025). An investigation of prompt variations for zero-shot LLM-based rankers. Retrieved from https://arxiv.org/abs/2406.14117.

Takehi, R. (2025). LLM-assisted relevance assessments: When should we ask LLMs for help? Retrieved from https://arxiv.org/abs/2411.06877.

Tang, R., Zhang, C., Ma, X., Lin, J., & Ture, F. (2024). Found in the middle: permutation self-consistency improves listwise ranking in large language models. In K. Duh, H. Gomez, & S. Bethard (Eds.), *Proceedings of the 2024 conference of the North American chapter of the Association for Computational Linguistics: human language technologies (volume 1: long papers)* (pp. 2327–2340). Mexico City, Mexico: Association for Computational Linguistics, http://dx.doi.org/10.18653/v1/2024.naacl-long.129, https://aclanthology.org/2024.naacl-long.129/.

Thakur, N., Reimers, N., Rücklé, A., Srivastava, A., & Gurevych, I. (2021). BEIR: a heterogeneous benchmark for zero-shot evaluation of information retrieval models. In *Thirty-fifth conference on neural information processing systems datasets and benchmarks track (round 2)*. https://openreview.net/forum?id=wCu6T5xFjeJ.

Upadhyay, S., Pradeep, R., Thakur, N., Campos, D., Craswell, N., Soboroff, I., et al. (2024). A large-scale study of relevance assessments with large language models: An initial look. Retrieved from https://arxiv.org/abs/2411.08275.

Yan, L., Qin, Z., Zhuang, H., Jagerman, R., Wang, X., Bendersky, M., et al. (2024). Consolidating ranking and relevance predictions of large language models through post-processing. In Y. Al-Onaizan, M. Bansal, Y.-N. Chen (Eds.), *Proceedings of the 2024 conference on empirical methods in natural language processing* (pp. 410–423). Miami, Florida, USA: Association for Computational Linguistics, http://dx.doi.org/10.18653/v1/2024.emnlp-main.25, https://aclanthology.org/2024.emnlp-main.25/.

Yuan, X. (2024). Improving zero-shot LLM re-ranker with risk minimization. In Y. Al-Onaizan, M. Bansal, & Y.-N. Chen (Eds.), *Proceedings of the 2024 conference on empirical methods in natural language processing* (pp. 17967–17983). Miami, Florida, USA: Association for Computational Linguistics, http://dx.doi.org/10.18653/v1/2024.emnlp-main.997, https://aclanthology.org/2024.emnlp-main.997/.

Zhai, C. (2024). Large language models and future of information retrieval: opportunities and challenges. In *Proceedings of the 47th international ACM SIGIR conference on research and development in information retrieval* (pp. 481–490).

Zhang, C., Hofstätter, S., Lewis, P., Tang, R., & Lin, J. (2025). Rank-without-gpt: building gpt-independent listwise rerankers on open-source large language models. In *Advances in information retrieval: 47th European conference on information retrieval, ECIR 2025, Lucca, Italy, April 6–10, 2025, proceedings, part II* (pp. 233–247). Berlin, Heidelberg: Springer-Verlag, ISBN: 978-3-031-88710-9, http://dx.doi.org/10.1007/978-3-031-88711-6_15.

Zhang, X., Yu, B., Yu, H., Lv, Y., Liu, T., Huang, F., et al. (2023). Wider and deeper LLM networks are fairer LLM evaluators. Retrieved from https://arxiv.org/abs/2308.01862.

Zhu, Y., Yuan, H., Wang, S., Liu, J., Liu, W., Deng, C., et al. (2024). Large language models for information retrieval: A survey. Retrieved from https://arxiv.org/abs/2308.07107.

Zhuang, H., Qin, Z., Hui, K., Wu, J., Yan, L., Wang, X., et al. (2024). Beyond yes and no: improving zero-shot LLM rankers via scoring fine-grained relevance labels. In K. Duh, H. Gomez, S. Bethard (Eds.), *Proceedings of the 2024 conference of the North American chapter of the Association for Computational Linguistics: Human language technologies (volume 2: short papers)* (pp. 358–370). Mexico City, Mexico: Association for Computational Linguistics, http://dx.doi.org/10.18653/v1/2024.naacl-short.31, https://aclanthology.org/2024.naacl-short.31/.

Zhuang, S., Zhuang, H., Koopman, B., & Zuccon, G. (2024). A setwise approach for effective and highly efficient zero-shot ranking with large language models. In *SIGIR 2024, Proceedings of the 47th international ACM SIGIR conference on research and development in information retrieval* (pp. 38–47). ACM, http://dx.doi.org/10.1145/3626772.3657813.